Memoria Práctica 1 - Pizzería

Estructuras de Datos

Autoras:

Ekaterina Nováková

Lía de Miguel Pastor - DNI: 34367884L

Curso Académico: 2026/2027

Estructuras de Datos **0**
1. Descripción de los TAD implementados y de las estructuras de datos definidas **1**
1.1 Estructuras básicas (structs y clases) 1
1.2 Tipos Abstractos de Datos (TADs) 1
2. Explicación del funcionamiento del simulador y de las funciones implementadas **2**
2.1 Generación de datos y aleatoriedad 2
2.2 Auxiliares para contar y mostrar datos 2
2.3 Lógica principal de la simulación 3
3. Problemas encontrados durante el desarrollo de la práctica y soluciones **4**
3.1 Recorrer Pilas y Colas sin destruir los nodos 4
3.2 Cancelar un pedido que está en mitad de una Pila 4
3.3 Control de errores al introducir datos en el menú por consola 5

Universidad de Alcalá

1. Descripción de los TAD implementados y de las estructuras de datos definidas

Para montar la simulación de la pizzería, hemos estructurado la práctica usando memoria dinámica con nodos enlazados (punteros). Definimos las estructuras básicas y los TADs principales en el cabecera **pizzeria4.h** para gestionar todo el flujo del programa.

1.1 Estructuras básicas (structs y clases)

- **struct Pedido**: Guarda los datos de cada pizza individual: **id_pedido** (string), **id_cliente** (int), **tipo_pizza** (string), **tamano** (string), **zona_reparto** (string), si es personalizada (bool) y el estado (string).

- **class Nodo**: Es la clase para gestionar la memoria dinámica. Contiene un objeto **Pedido** y un puntero **siguiente** que apunta al siguiente nodo de la lista. Incluye un constructor con valores por defecto para simplificar la creación de nuevos nodos.

- **typedef Nodo *pNodo**: Un alias para simplificar el código al manejar punteros a **Nodo**, haciendo la lectura del código mucho más limpia.

1.2 Tipos Abstractos de Datos (TADs)

Para ocultar la gestión interna de punteros y proteger los datos, creamos dos clases principales:

- Clase Cola (TAD Cola):
  - Funcionamiento: Es un FIFO (First In, First Out). El primer pedido en entrar a la cola es el primero que se atiende.
  - Atributos: Mantiene dos punteros: primero (para desencolar por el frente) y ultimo (para encolar por el final).
  - Uso en la práctica: Simula las fases por las que pasa una pizza: cola de pedidos recibidos, **preparacionEspecial**, **cocina**, las 4 colas de expedición por zonas (Norte, Sur, Este, Oeste) y el historial de pedidos finalizados.
  - Métodos principales: **encolar()**, **desencolar()**, **valorFrente()** y **esVacia()**.
- Clase Pila (TAD Pila):
  - Funcionamiento: Es un LIFO (Last In, First Out). El último pedido guardado es el primero en salir.
  - Atributos: Mantiene únicamente el puntero cima, que apunta siempre al nodo superior de la pila.
  - Uso en la práctica: Se usa para las mochilas térmicas de las motos de reparto (**cajaNorte**, **cajaSur**, etc.). Refleja la realidad física: la última pizza que metes en la caja del repartidor queda arriba y es la primera que se saca.
  - Métodos principales: **apilar()**, **desapilar()**, **valorCima()** y **esVacia()**.

2. Explicación del funcionamiento del simulador y de las funciones implementadas

2.1 Generación de datos y aleatoriedad

- **int calcularSecuenciaAleatoria(int x)**
  - Funcionamiento: Devuelve un número entero aleatorio usando el operador módulo (**rand() % x**).
- **string generarIdPedido(int contador)**
  - Funcionamiento: Genera un ID único para cada pedido sumando un contador incremental a la base 10000 y poniéndole el prefijo "P" (por ejemplo, "P10001", "P10002"), así es fácil hacer seguimiento de las pizzas.
- **Pedido generarPedidoAleatorio(int &contadorGlobal)**
  - Funcionamiento: Crea y devuelve una estructura **Pedido** con datos generados al azar. Elige tamaño, masa y zona usando arrays de strings y la función aleatoria, y determina si la pizza es personalizada usando la constante **PROB_PERSONALIZACION**. Además, va incrementando el **contadorGlobal** (pasado por referencia) para ir variando las IDs.

2.2 Auxiliares para contar y mostrar datos

Como las Pilas y Colas son estructuras opacas y solo nos dejan acceder a la cima o al frente, para no perder la información al recorrerlas hacemos un vaciado temporal y luego las restauramos:

- **int contarCola(Cola &c)** y **int contarCaja(Pila &caja)**
  - Funcionamiento: Cuentan cuántos elementos hay guardados. Van sacando los nodos uno a uno (con **desencolar()** o **desapilar()**), los meten en una estructura auxiliar local (aux) y suman 1 al contador. Al terminar, vuelven a pasar los datos de la auxiliar a la original para dejarlo todo tal cual estaba.
- **void mostrarCola(Cola &c, const string &nombreCola)** y **void mostrarCaja(Pila &caja)**
  - Funcionamiento: Imprimen en consola los elementos del contenedor de forma clara. Hacen la misma jugada de vaciar en una estructura auxiliar y volver a rellenar la original. En las colas se muestran los datos alineados en formato tabla (usando **setw**), y en las pilas se imprime desde la cima hacia abajo para simular visualmente cómo están apiladas las pizzas dentro de la caja.

2.3 Lógica principal de la simulación

- **bool retirarPedido(Pila &caja, const string &idPedido, Cola &finalizados)**
  - Funcionamiento: Sirve para cancelar o retirar un pedido que esté metido en la caja térmica del repartidor. Va desapilando en una pila auxiliar hasta dar con la pizza que coincide con **idPedido**. Cuando la encuentra, le cambia el estado a "Retirado", la pasa a la cola de finalizados y devuelve las demás pizzas a la caja original sin cambiar su orden. Devuelve **true** si lo encontró o **false** si no estaba.
- **void mostrarResumenCajas(...)**
  - Funcionamiento: Muestra un resumen del estado del sistema en pantalla. Llama a **contarCaja()** para ver qué tan llenas están las mochilas de las 4 motos según el máximo permitido (**CAP_CAJA**), y suma las motos enviadas para mostrar las estadísticas en tiempo real.
- **void procesarCaja(Pila &caja, Cola &expedicionZona, int cajasEnviadas[], Cola &finalizados, int zona)**
  - Funcionamiento: Pasa los pedidos listos de la cola de expedición a la mochila térmica del repartidor, cambiando su estado a "En caja". Si la caja se llena hasta el límite (**CAP_CAJA = 6** pizzas), se envía la moto: desapila las pizzas cambiándoles el estado a "Enviado", las mete en la cola de finalizados e incrementa en 1 los viajes de esa zona en el vector **cajasEnviadas**.
- **void ejecutarPasos(...)**
  - Funcionamiento: Es el motor que hace avanzar la simulación en bloques de **N_PEDIDOS_PASO** (10 pedidos por turno) para imitar el trabajo real de la pizzería. Mueve de forma ordenada el flujo: llena cajas, pasa de cocina a expedición, de **preparacionEspecial** a cocina, y clasifica los pedidos de la cola de recibidos según si son personalizados o normales.

3. Problemas encontrados durante el desarrollo de la práctica y soluciones

3.1 Recorrer Pilas y Colas sin destruir los nodos

Problema: Los TADs que implementamos solo nos dejan consultar el frente (colas) o la cima (pilas). Al no tener activado el constructor de copia (**= delete**), no podíamos sacar una copia directa de la estructura para imprimirla o contar cuántos elementos tenía. Si desencolábamos o desapilábamos directamente para contar o mostrar por pantalla, la estructura original se quedaba vacía y perdíamos la lista de nodos enlazados en memoria.

Solución: Para arreglar esto, creamos una estructura auxiliar en la propia función. Vamos sacando los elementos de la estructura original, guardamos los datos necesarios o imprimimos en pantalla, y los metemos temporalmente en la auxiliar. Una vez terminamos de recorrerla entera, hacemos el proceso inverso volcando todo de vuelta a la estructura inicial, manteniendo intactos el orden y los punteros.

3.2 Cancelar un pedido que está en mitad de una Pila

Problema: Al usar el TAD Pila para las mochilas de las motos (comportamiento LIFO), si un cliente cancela un pedido que está metido en medio o al fondo de la mochila, la propia estructura no permite acceder directamente a ese nodo sin quitar primero las pizzas que tiene encima.

Solución: Lo resolvimos usando una pila auxiliar durante la retirada en la función **retirarPedido**. Desapilamos los elementos superiores y los dejamos en la pila temporal hasta llegar a la pizza deseada (comprobando el **id_pedido** con **valorCima()**). La marcamos como "Retirado", la mandamos al historial de finalizados y devolvemos las pizzas guardadas en la auxiliar a la mochila original, respetando la pila original sin romper el orden del resto.

3.3 Control de errores al introducir datos en el menú por consola

Problema: Cuando en el menú de **main.cpp** se esperaba un entero para seleccionar una opción y el usuario introducía por error una letra o carácter no válido, el flujo de entrada (**cin**) entraba en estado de fallo. El carácter no válido se quedaba colgado en el búfer de entrada, provocando un bucle infinito en el **switch** que congelaba el programa.

Solución: Añadimos una validación para controlar el error del flujo preguntando con **if (!(cin >> opcion))**. Si falla, limpiamos los flags de error del flujo con **cin.clear()** y descartamos los caracteres no válidos del búfer de entrada usando **cin.ignore(1000, '\n')**. De esta forma el menú vuelve a pedir la opción de forma limpia sin colgarse.